Fundamentals of Programming using Python • First Year, FinTech

Chapter 02: Where You Write It: Interpreter, Scripts, Notebooks, IDEs

The three ways to run Python • the edit–run–read cycle • notebooks and their trap • choosing an IDE • local or cloud

Chapter 02 of 10

Dr. Brahim Zirari

Yahia Farès University of Medea

in linkedin.com/in/brahimzirari · Google Scholar

October 2026

1. Three ways to run Python

❑ **The same language, three doors** – Chapter 01 showed you what to write. This chapter is about *where* you write it. Python itself does not change: `print(250000 * 0.035)` behaves identically in all three. What changes is how much you keep, and how easily you can run it again.

2. The interpreter

❑ **What it is** – The **interpreter** — also called the *REPL*, for Read, Evaluate, Print, Loop — is Python waiting for you to type one line. You type, press Enter, and it answers immediately. Its prompt is three angle brackets: `>>>`.

❑ **How to start it** – Open a terminal and type `python` (on some systems `python3`), then Enter. To leave, type `exit()` or press Ctrl+D.

❑ **What it is good for** – Checking one thing. *Does `7 // 2` give 3? What type is `7 / 1`? What happens if I add a string to a number?* It answers in two seconds, and that is exactly the right tool for such questions.

Pitfall: nothing is saved. Close the window and the whole session is gone — your variables, your calculations, everything. Never do real work here. The interpreter is for asking, not for building.

3. Scripts

❑ **What a script is** – A **script** is a file with the extension `.py` containing Python lines in the order you want them run. Python reads it from top to bottom and executes every line. It is the normal way to write a program.

❑ **Three rules for the file** –

- The name ends in `.py` — not `.txt`, not `.py.txt`. On Windows, turn on "show file extensions" or you will not see the mistake.
- No spaces and no accents in the name: `bank_fees.py`, not `bank fees.py` or `frais_banqueé.py`.
- Do not name a file after something Python already knows: a file called `math.py` or `random.py` will break programs that try to use the real library of that name.

❑ **How to run it** – From the terminal, in the folder containing the file:

Terminal

```
python fees.py
```

Remark: or press the Run button in your editor, which does exactly this for you and shows the output in a panel.

4. The edit–run–read cycle

❑ **This is the whole job** – Writing a program is not typing it once correctly. It is going round this cycle, often dozens of times for one small program. Going round it is not failure; it *is* programming.

*Pitfall: **Save before you run.** Most editors run the file* on disk*, not the text on your screen. An unsaved change means you are running the old version and reading output that does not match the code you are looking at — one of the most confusing bugs there is. Ctrl+S, every time.*

❑ **Change one thing at a time** – When an error appears, change one thing, run, and look. Changing three things at once means you will not know which one helped.

5. Notebooks

❑ **What a notebook is** – A **notebook** is a document made of **cells**. A code cell holds Python and shows its output directly underneath; a text cell holds explanation. You run one cell at a time with Shift+Enter. The file extension is `.ipynb`.

❑ **Why they are popular in data work** – The output sits next to the code that produced it, so a table or a chart appears exactly where it is discussed. A finished notebook reads like a report that can be re-run.

❑ **The counter** – Each time a cell runs, its number increases: `In [1]`, `In [2]`, `In [3]`. **That number is the order in which cells were run, not the order in which they appear.** A cell that has never run shows `In [ ]`.

6. The notebook trap

❑ **Out-of-order execution** – Because you may run cells in any order, and re-run them, the screen can show an answer that the code no longer produces.

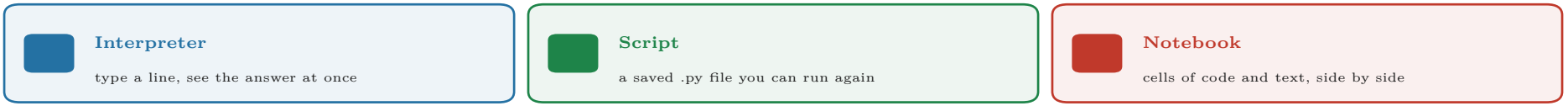


Figure 1. The three ways to run Python. You will use all three, for different jobs.

Pitfall: this is the single commonest source of wrong results in data work, and it produces no error. *The number is simply wrong, and it looks fine.*

❑ **The cure** – Before you trust a notebook — and always before you hand one in — choose **Restart kernel and run all cells**. If it does not produce the same results from a clean start, the results were never real.

7. Script or notebook?

	Script (`.py`)	Notebook (`.ipynb`)
Runs	top to bottom, always	cell by cell, in any order
Output	in the terminal	under each cell, kept in the file
Best for	a program you run again: a report, a job, a tool	exploring data, explaining a result, teaching
Re-reading	the file is the truth	the file may show stale output
Sharing	plain text, easy to compare versions	a large file; versions are hard to compare

❑ **The practical answer** – Explore in a notebook. When the work is right, move it into a script. In this course the TP exercises are handed in as scripts, because a script is the thing that can be checked by running it.

8. What an IDE adds

❑ **Definition** – An **IDE** — Integrated Development Environment — is an editor with the programmer's tools built in. It does not change Python. It changes how quickly you find your mistakes.

❑ **The one that matters most** – The **debugger**. It stops the program on a chosen line and lets you look at the value of every variable at that moment. Printing values to find a bug works; stepping through with a debugger works faster.

9. The main environments

❑ **What to choose for this course** – **Any one of them.** The exercises are small, and all of these run them. If you have no preference: **Thonny** if you have never programmed before, **VS Code** if you intend to keep programming, **Google Colab** if installing anything on your machine is a problem.

Remark: the editor is not the skill. A student who knows Python can move to a new editor in an afternoon; a student who knows only one editor's buttons has learned nothing portable. Do not spend the first week choosing.

10. Local or cloud?

	Local (your machine)	**Cloud** (the browser)
Install	Python and an editor, once	nothing
Works offline	yes	no
Your files	on your disk, yours	on someone's server
Power	your machine	often more, sometimes a free GPU
Risk	an installation that goes wrong	no internet, no work; a session that stops and loses everything

Pitfall: a cloud session is temporary. Google Colab disconnects after a period of inactivity and everything not saved to your own drive is lost — including any file you uploaded. Download your work before you close the tab.

11. Installing Python

❑ **Two routes** – Install **Python from python.org** and add an editor separately, or install **Anaconda**, which brings Python together with Spyder, Jupyter and the main data libraries in one download. The first is smaller and gives you control; the second saves you from installing libraries one by one.

❑ **One thing not to miss** – On Windows, the installer shows a checkbox reading **"Add python.exe to PATH"**. Tick it. If you do not, the terminal will not know what `python` means and you will meet the error below on your first attempt.

❑ **The three first-day errors** –

`'python' is not recognised`	Python is not on PATH. Re-run the installer and tick the box, or use the editor's own Run button.
`can't open file 'fees.py'`	the terminal is not in the folder containing the file. Move to that folder first, or open the folder in your editor and use Run.
`SyntaxError` *on a correct line*	the file was saved as `.txt`, or an old Python 2 line is above the one named.

12. Your first file, start to finish

1. Make a folder for this course — for example `fintech_python`.
2. Open that *folder* in your editor, not just the file. The editor then knows where your files are, and so does its terminal.
3. Create a new file called `fees.py`.
4. Type the four lines below. Type them; do not paste them.
5. Save (Ctrl+S), then Run.
6. Read the output. If instead you get a traceback, read its last line first.

The interpreter: every line answered immediately, nothing kept.

```
>>> 250000 * 0.035
8750.0
>>> capital = 250000
>>> capital * 0.04
10000.0
>>> exit()
```

Figure 2. An interpreter session. You do not type the »» — Python prints it.

```
# first file - banking fee report
fee = 1250
vat = fee * 0.19
print(f"Fee {fee} DZD, VAT {vat:.2f} DZD, "
      f"total {fee + vat:.2f} DZD")
```

Output

```
Fee 1250 DZD, VAT 237.50 DZD, total 1487.50 DZD
```

Remark: two strings written one after the other are joined by Python, which is why the last two lines above produce a single line of output. It is the usual way to keep a long `print` inside the width of the screen.

Quick reference

Running Python	
`python`	start the interpreter (`python3` on some systems)
`exit()`	leave the interpreter
`python fees.py`	run a script, from its own folder
`»`	the interpreter's prompt. You never type it
Files	
`.py`	a script. Plain text, runs top to bottom
`.ipynb`	a notebook. Cells, run in any order
In a notebook	
Shift+Enter	run this cell
`In [7]`	this cell was the 7th *run*, not the 7th written
`In [ ]`	this cell has never run
Restart & run all	the only way to trust a notebook
Everywhere	
Ctrl+S	save. The editor runs the file on disk, not your screen

What you can do after this chapter

- Name the three ways to run Python and say what each is for.
- Start the interpreter, use it to check one thing, and leave it.
- Create, save and run a `.py` script.
- Describe the edit–run–read cycle, and why saving comes before running.
- Explain the out-of-order trap in notebooks and the one cure for it.
- Choose between a script and a notebook for a given job.
- Install Python, or choose a cloud alternative, and recognise the three first-day errors.

❑ **Next** – Chapter 03 — back to the language: variables and the basic commands, in full.

↪ downloads: `www.python.org/downloads/`

↪ Anaconda: `www.anaconda.com/download`

↪ Google Colab: `colab.research.google.com/`

↪ VS Code for Python: `code.visualstudio.com/docs/languages/python`

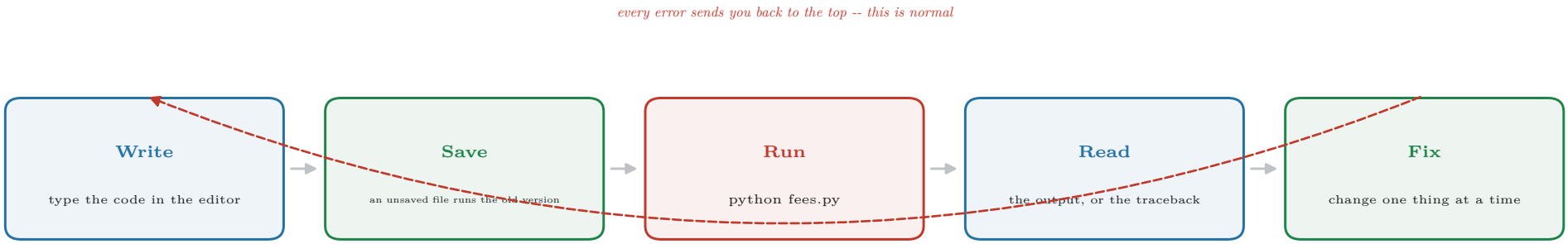


Figure 3. The cycle you will repeat all year.

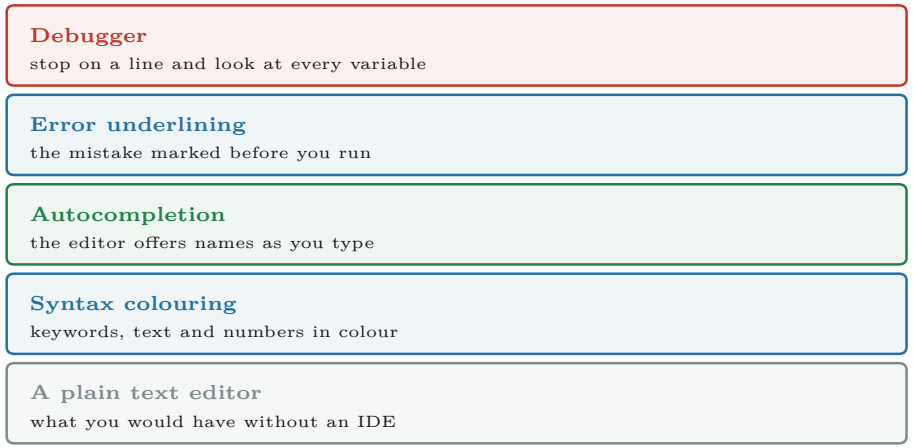


Figure 5. Read from the bottom: each layer is something you would otherwise do by hand.



Figure 6. The tools you will meet. Anaconda is listed with them but is a distribution, not an editor.